# Software Requirements Specification (SRS)

**Project:** IT Resource & Operations Platform (IROP) **Version:** 1.0 **Date:** November 2026 **Author:** IROP Development Team **Classification:** Internal ΓÇö Technical Specification

## Table of Contents

1. Introduction
2. Functional Requirements
3. Non-Functional Requirements
4. Business Rules
5. Data Requirements
6. Interface Requirements
7. Constraints

## 1. Introduction

### 1.1 Purpose

This Software Requirements Specification defines the complete set of functional and non-functional requirements for the IT Resource & Operations Platform (IROP). It serves as the authoritative reference for development, testing, and review.

### 1.2 Scope

IROP is a full-stack web application comprising three operational modules ΓÇö Leave Management, Asset Management, and Inventory Management ΓÇö unified under a single role-based access control system. The application targets internal use within an IT company.

### 1.3 Definitions & Abbreviations

| Term | Definition |
|---|---|
| JWT | JSON Web Token ΓÇö a compact, URL-safe token format for authentication |
| RBAC | Role-Based Access Control |
| LMS | Leave Management System |
| AMS | Asset Management System |
| IMS | Inventory Management System |
| SPA | Single Page Application |
| CRUD | Create, Read, Update, Delete |
| OWASP | Open Web Application Security Project |

### 1.4 References

- PRD.md ΓÇö Product Requirements Document
- API.md ΓÇö REST API Specification
- DOCKER_CICD_RND.md ΓÇö Docker & CI/CD Research & Development Report
- DECISIONS.md ΓÇö Architecture Decisions & Design Rationale
- TEST_REPORT.md ΓÇö Verification & Test Report

### 1.5 Technology Architecture: MERN Stack Selection & Justification

The technical foundation of IROP is built upon the **MERN Stack (MongoDB, Express.js, React 18, Node.js)** combined with **TypeScript**. The engineering team selected this stack based on the specific operational requirements of this platform:

1. **MongoDB (Document Database)**:
    - **Polymorphic IT Assets**: Hardware items (laptops, monitors) possess physical attributes such as serial numbers and warranty dates, while software licenses require digital license keys and seat limits. MongoDB's schema-flexible BSON model natively handles diverse asset types without unwieldy multi-table SQL joins.
    - **Atomic Quota & Stock Modifications**: Mongoose integration with MongoDB's `$inc` operator guarantees race-condition-free leave balance deductions and inventory stock adjustments under concurrent user operations.
    - **Nested Organization Documents**: Quotas ( `casual` , `sick` , `annual` , `wfh` ) and reporting hierarchy links are stored directly within user documents for single-query retrieval performance.
2. **Express.js 5 & Node.js 20 (REST API Engine)**:
    - **Asynchronous Throughput**: Node's non-blocking I/O efficiently serves concurrent requests across leave applications, asset filtering, and inventory tracking with minimal server memory footprint.
    - **Modular Middleware Pipeline**: Express enables clean composability of enterprise security middleware: `helmet` for HTTP response hardening, `cors` for trusted origin enforcement, `express-rate-limit` for brute-force mitigation, and JWT verification filters.
3. **React 18 & Vite (Interactive SPA Frontend)**:

- **Dynamic Operational Interface**: IT operations require snappy, real-time feedback ΓÇö searching through assets with debounced filters, one-click +/- inventory counters, and dynamic weekday calculations during leave requests. React's virtual DOM and state management eliminate jarring page reloads.
- **Client State & Cache Layering**: TanStack Query handles caching, background revalidation, and optimistic mutations, significantly decreasing network overhead.

4. **Full-Stack Strict TypeScript**:
   - Sharing interfaces and Zod validation schemas between the REST API and the React client guarantees end-to-end type safety, preventing runtime null pointer crashes and eliminating contract mismatches.

---

## 2. Functional Requirements

Requirements are numbered per module. Priority levels: **M** = Must Have, **S** = Should Have, **C** = Could Have.

### 2.1 Authentication (AUTH)

| ID | Requirement | Priority |
|---|---|---|
| FR-AUTH-1 | The system SHALL provide a `POST /api/auth/login` endpoint that accepts `email` and `password`, validates credentials against bcrypt-hashed passwords, and returns a signed JWT containing `id` and `role` in the payload. | M |
| FR-AUTH-2 | The system SHALL reject login attempts with a generic "invalid credentials" message, never revealing whether the email or password was incorrect. | M |
| FR-AUTH-3 | The system SHALL prevent inactive users ( `isActive = false` ) from logging in, returning the same generic error message. | M |
| FR-AUTH-4 | The system SHALL provide a `GET /api/auth/me` endpoint that returns the authenticated user's profile (excluding `passwordHash` ). | M |
| FR-AUTH-5 | The system SHALL provide a `POST /api/auth/change-password` endpoint that accepts `currentPassword` and `newPassword`, validates the current password, and updates the hash. | S |
| FR-AUTH-6 | The system SHALL NOT provide a public signup/registration endpoint. Only admins create user accounts. | M |
| FR-AUTH-7 | The system SHALL require a valid `Authorization: Bearer <token>` header on all protected endpoints. Requests without a valid token SHALL receive `401 Unauthorized` . | M |
| FR-AUTH-8 | The system SHALL use bcrypt with a cost factor ΓëÑ 10 for all password hashing operations. | M |
| FR-AUTH-9 | The system SHALL apply `requireAuth` middleware followed by `requireRole(...roles)` middleware on every protected route. | M |
| FR-AUTH-10 | The system SHALL apply rate limiting on the login endpoint to mitigate brute-force attacks. | S |

### 2.2 User Management (USER)

| ID | Requirement | Priority |
|---|---|---|
| FR-USER-1 | The system SHALL provide a `POST /api/users` endpoint (admin only) to create new users with fields: `name`, `email`, `password`, `role`, `department`, `managerId` (optional), and `quotas`. | M |
| FR-USER-2 | The system SHALL enforce unique, case-insensitive email addresses across all user accounts. | M |
| FR-USER-3 | The system SHALL support three roles: `employee`, `manager`, and `admin`. | M |
| FR-USER-4 | The system SHALL provide a `GET /api/users` endpoint (admin only) that returns a paginated list of all users. | M |
| FR-USER-5 | The system SHALL provide a `PATCH /api/users/:id` endpoint (admin only) to update a user's `role`, `managerId`, `department`, `quotas`, and `isActive` status. | M |
| FR-USER-6 | The system SHALL store a `managerId` reference on each user to establish a team hierarchy for leave approvals. | M |
| FR-USER-7 | The system SHALL initialise each user with default quotas: `{ casual: { total, used: 0 }, sick: { total, used: 0 }, annual: { total, used: 0 }, wfh: { total, used: 0 } }`. | M |
| FR-USER-8 | The system SHALL provide a password reset endpoint (admin only) to set a new password for any user. | S |
| FR-USER-9 | The system SHALL never return `passwordHash` in any API response. | M |
| FR-USER-10 | The system SHALL prevent the last remaining admin from being deactivated or demoted to a non-admin role. | M |

### 2.3 Leave Management (LEAVE)

| ID | Requirement | Priority |
|---|---|---|
| FR-LEAVE-1 | The system SHALL provide a `POST /api/leaves` endpoint for authenticated users to submit a leave request with fields: `type` (casual / sick / annual / wfh), `startDate`, `endDate`, and `reason`. | M |
| FR-LEAVE-2 | The system SHALL compute the `days` field as the count of weekdays (MondayΓÇôFriday) between `startDate` and `endDate` inclusive. | M |
| FR-LEAVE-3 | The system SHALL validate that `endDate >= startDate` and that `startDate` is not in the past for new requests, with the exception that WFH may start today. | M |
| FR-LEAVE-4 | The system SHALL reject a leave request if the user has an existing pending or approved leave request with overlapping dates. | M |
| FR-LEAVE-5 | The system SHALL reject a leave request if the remaining quota for the specified type is less than the computed `days`. | M |
| FR-LEAVE-6 | The system SHALL create the leave request with status `pending`. Quota is NOT deducted at creation time. | M |
| FR-LEAVE-7 | The system SHALL provide a `GET /api/leaves/mine` endpoint that returns all leave requests for the authenticated user, sorted by date descending, with pagination. | M |
| FR-LEAVE-8 | The system SHALL provide a `PATCH /api/leaves/:id/cancel` endpoint that allows a user to cancel their own pending leave request (sets status to `cancelled`; no quota deduction). | M |
| FR-LEAVE-9 | The system SHALL provide a `GET /api/leaves/team` endpoint: for managers, returns pending requests from their direct reports; for admins, returns all pending requests. | M |
| FR-LEAVE-10 | The system SHALL provide a `PATCH /api/leaves/:id/approve` endpoint that sets status to `approved`, atomically deducts the `days` from the user's quota for that leave type, and records `reviewedBy`, `reviewedAt`, and optional `reviewNote`. | M |
| FR-LEAVE-11 | The system SHALL provide a `PATCH /api/leaves/:id/reject` endpoint that sets status to `rejected` with `reviewedBy`, `reviewedAt`, and optional `reviewNote`. No quota is deducted. | M |
| FR-LEAVE-12 | The system SHALL ensure that only `pending` leave requests can be approved or rejected. Attempting to approve/reject a non-pending request SHALL return an error. | M |
| FR-LEAVE-13 | A manager SHALL only be able to approve or reject leave requests from users whose `managerId` matches the manager's ID. An admin SHALL be able to approve or reject any user's leave. | M |
| FR-LEAVE-14 | The system SHALL prevent any user from approving or rejecting their own leave request. | M |
| FR-LEAVE-15 | The system SHALL provide a `GET /api/leaves/balance` endpoint that returns the authenticated user's quota for each leave type: `{ total, used, remaining }`. | M |

## 2.4 Asset Management (ASSET)

| ID | Requirement | Priority |
|---|---|---|
| FR-ASSET-1 | The system SHALL provide a `POST /api/assets` endpoint (admin only) to create an asset with fields: `name`, `kind` (hardware / software), `serialNumber`, `status`, `assignedTo` (optional), `purchaseDate` (optional). | M |
| FR-ASSET-2 | The system SHALL enforce unique serial numbers (sparse index: null is allowed, but non-null values must be unique). | M |
| FR-ASSET-3 | The system SHALL provide a `GET /api/assets` endpoint: admin and manager see all assets; employees see only their own assigned assets. | M |
| FR-ASSET-4 | The `GET /api/assets` endpoint SHALL support filtering by `status` (in_use / available / in_repair) via query parameter. | M |
| FR-ASSET-5 | The `GET /api/assets` endpoint SHALL support full-text search via a `q` query parameter, searching across `name` and `serialNumber` fields. | M |
| FR-ASSET-6 | The system SHALL provide a `GET /api/assets/mine` endpoint that returns only assets assigned to the authenticated user. | S |
| FR-ASSET-7 | The system SHALL provide a `PATCH /api/assets/:id` endpoint (admin only) to update asset details. | M |
| FR-ASSET-8 | The system SHALL provide a `DELETE /api/assets/:id` endpoint (admin only) to remove an asset. | M |
| FR-ASSET-9 | The system SHALL provide a `PATCH /api/assets/:id/assign` endpoint (admin only) that sets `assignedTo` to a specified user and changes `status` to `in_use`. | M |
| FR-ASSET-10 | The system SHALL provide a `PATCH /api/assets/:id/unassign` endpoint (admin only) that clears `assignedTo` and sets `status` to `available`. | M |
| FR-ASSET-11 | The system SHALL reject assignment of an asset that has status `in_repair`. | M |
| FR-ASSET-12 | The `GET /api/assets` endpoint SHALL support pagination via `page` and `limit` query parameters. | S |

### 2.5 Inventory Management (INVENTORY)

| ID | Requirement | Priority |
|---|---|---|
| FR-INV-1 | The system SHALL provide a `POST /api/inventory` endpoint (admin only) to create an inventory item with fields: `name` (unique), `category`, `quantity` (ГëÑ 0), `threshold` (ГëÑ 0), and `unit`. | M |
| FR-INV-2 | The system SHALL provide a `GET /api/inventory` endpoint accessible to admin and manager roles. Employees SHALL NOT access inventory. | M |
| FR-INV-3 | The system SHALL provide a `PATCH /api/inventory/:id` endpoint (admin only) to update inventory item details. | M |
| FR-INV-4 | The system SHALL provide a `DELETE /api/inventory/:id` endpoint (admin only) to remove an inventory item. | M |
| FR-INV-5 | The system SHALL provide a `PATCH /api/inventory/:id/adjust` endpoint (admin only) that accepts `{ delta: +1 | -1 } and atomically adjusts the item's quantity`. |
| FR-INV-6 | The system SHALL reject any adjustment that would reduce `quantity` below 0. | M |
| FR-INV-7 | The system SHALL compute a virtual `isLowStock` property that is `true` when `quantity <= threshold`. | M |
| FR-INV-8 | Manager role SHALL have read-only access to inventory (no create, update, delete, or adjust). | M |
| FR-INV-9 | The `GET /api/inventory` endpoint SHALL support pagination via `page` and `limit` query parameters. | S |

### 2.6 Health Check (HEALTH)

| ID | Requirement | Priority |
|---|---|---|
| FR-HEALTH-1 | The system SHALL provide a `GET /health` endpoint (no authentication required) that returns `{ status: "ok", db: "connected" | "disconnected", uptime: }`. |

---

## 3. Non-Functional Requirements

### 3.1 Performance

| ID | Requirement |
|---|---|
| NFR-PERF-1 | API responses for single-resource operations (create, read, update, delete) SHALL complete within 500ms under normal load. |
| NFR-PERF-2 | Paginated list endpoints SHALL return results within 1 second for collections of up to 10,000 records. |
| NFR-PERF-3 | The client application SHALL render the initial view within 3 seconds on a standard broadband connection. |
| NFR-PERF-4 | Asset search with text indexing SHALL return results within 2 seconds. |

## 3.2 Security

| ID | Requirement |
|---|---|
| NFR-SEC-1 | All passwords SHALL be hashed using bcrypt with a cost factor ГёÑ 10. Plain-text passwords SHALL never be stored or logged. |
| NFR-SEC-2 | Password hashes SHALL never be included in any API response. |
| NFR-SEC-3 | The server SHALL use Helmet.js to set security-related HTTP headers (X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, etc.). |
| NFR-SEC-4 | The login endpoint SHALL be rate-limited (e.g., `express-rate-limit`) to mitigate brute-force attacks. |
| NFR-SEC-5 | JWT secrets SHALL be stored in environment variables, never hardcoded or committed to version control. |
| NFR-SEC-6 | CORS SHALL be configured to allow only the expected client origin (`CLIENT_ORIGIN` env var). |
| NFR-SEC-7 | Error responses SHALL never expose stack traces, internal paths, or password hashes. |
| NFR-SEC-8 | The application SHALL use parameterised queries (Mongoose) to prevent NoSQL injection. |
| NFR-SEC-9 | Input SHALL be validated using Zod schemas on every mutating endpoint before processing. |
| NFR-SEC-10 | Docker containers SHALL run as non-root users. |
| NFR-SEC-11 | No secrets (passwords, tokens, connection strings) SHALL be embedded in Docker image layers. |

## 3.3 Usability

| ID | Requirement |
|---|---|
| NFR-USE-1 | The UI SHALL be responsive and usable on desktop (ГëÑ 1024px) and tablet (ГëÑ 768px) viewports. |
| NFR-USE-2 | Every data-fetching screen SHALL display appropriate loading, empty, and error states. |
| NFR-USE-3 | Status badges SHALL use consistent, accessible colours: pending (amber), approved (green), rejected (red), cancelled (grey). |
| NFR-USE-4 | Destructive actions (delete, reject) SHALL require confirmation dialogs. |
| NFR-USE-5 | Success and error feedback SHALL be delivered via toast notifications. |
| NFR-USE-6 | Forms SHALL have accessible labels, proper focus management, and keyboard-friendly controls. |

### 3.4 Reliability

| ID | Requirement |
|---|---|
| NFR-REL-1 | The health endpoint SHALL accurately reflect database connectivity status. |
| NFR-REL-2 | Quota deduction on leave approval SHALL be performed atomically with the status change to prevent inconsistencies. |
| NFR-REL-3 | The seed script SHALL be idempotent: running it multiple times SHALL produce identical data (8 users, 7 leaves, 10 assets, 8 inventory items). |
| NFR-REL-4 | The application SHALL handle MongoDB connection failures gracefully, returning appropriate error responses rather than crashing. |

### 3.5 Maintainability

| ID | Requirement |
|---|---|
| NFR-MAINT-1 | The codebase SHALL use TypeScript in strict mode throughout (both client and server). |
| NFR-MAINT-2 | The project SHALL pass ESLint checks with no errors. |
| NFR-MAINT-3 | Environment configuration SHALL be validated at startup using Zod schemas, failing fast with clear messages on missing/invalid variables. |

---

## 4. Business Rules

The following business rules SHALL be enforced on the server side. Client-side validation is a convenience only; the server is the authoritative enforcer.

| ID | Rule | Module |
|---|---|---|
| BR-1 | Leave `days` are computed as weekdays (MonΓÇôFri) between `startDate` and `endDate` inclusive. `endDate` must be ΓëÑ `startDate`. New requests (except WFH for today) must not have a past `startDate`. | LMS |
| BR-2 | A user cannot have overlapping pending or approved leave requests for the same date range. | LMS |
| BR-3 | A leave request SHALL be rejected if the remaining quota for that leave type is less than the computed `days`. WFH leave is included in quota tracking. | LMS |
| BR-4 | Quota is deducted **only** when a leave request transitions from `pending` to `approved`. The deduction is performed atomically with the status change. Rejected and cancelled requests never affect quota. | LMS |
| BR-5 | Only leave requests with status `pending` can be approved or rejected. A user SHALL never approve or reject their own leave request. | LMS |
| BR-6 | A manager can approve or reject leave only for users whose `managerId` matches the manager's ID. An admin can act on any user's leave (including managers'). | LMS |
| BR-7 | Inventory `quantity` cannot go below 0. An item is considered low-stock ( `isLowStock = true` ) when `quantity <= threshold`. | IMS |
| BR-8 | Assigning an asset sets its status to `in_use`. Unassigning sets it to `available`. An asset with status `in_repair` cannot be assigned. | AMS |
| BR-9 | An admin cannot deactivate or demote themselves if they are the last remaining admin in the system. | USER |

## 5. Data Requirements

### 5.1 User Schema

```
User {   _id:         ObjectId (auto-generated)   name:        String, required   email:       String, requi
```

### 5.2 LeaveRequest Schema

```
LeaveRequest {   _id:         ObjectId (auto-generated)   userId:      ObjectId (ref: User), required, indexe
```

### 5.3 Asset Schema

```
Asset {   _id:         ObjectId (auto-generated)   name:         String, required   kind:         Enum ['ha
```

### 5.4 InventoryItem Schema

```
InventoryItem {   _id:       ObjectId (auto-generated)   name:      String, required, unique   category:   St
```

### 5.5 Entity Relationships

```
ГöîГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÉ      1:N      ГöîГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÉ Гöé   U
```

## 6. Interface Requirements

### 6.1 REST API Interface

| Aspect | Specification |
|---|---|
| Protocol | HTTP/HTTPS |
| Base URL | `/api` (all module endpoints), `/health` (health check, no `/api` prefix) |
| Authentication | `Authorization: Bearer <JWT>` header on all protected endpoints |
| Content Type | `application/json` for requests and responses |
| Pagination | Query parameters `page` (1-based, default: 1) and `limit` (default: 10) on list endpoints |
| Error Format | `{ "error": { "code": "ERROR_CODE", "message": "Human-readable message", "details": [...] } }` |
| HTTP Status Codes | 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 409 (Conflict), 500 (Internal Server Error) |

### 6.2 API Endpoint Summary

| Method | Endpoint | Auth | Roles | Description |
|---|---|---|---|---|
| `POST` | `/api/auth/login` | No | Any | Authenticate user, return JWT |
| `GET` | `/api/auth/me` | Yes | Any | Get current user profile |
| `POST` | `/api/auth/change-password` | Yes | Any | Change own password |
| `GET` | `/api/users` | Yes | Admin | List all users (paginated) |
| `POST` | `/api/users` | Yes | Admin | Create a new user |
| `PATCH` | `/api/users/:id` | Yes | Admin | Update user (role, quotas, etc.) |
| `POST` | `/api/leaves` | Yes | Any | Submit a leave request |
| `GET` | `/api/leaves/mine` | Yes | Any | Get own leave requests |
| `GET` | `/api/leaves/balance` | Yes | Any | Get own leave balance |
| `PATCH` | `/api/leaves/:id/cancel` | Yes | Any | Cancel own pending leave |
| `GET` | `/api/leaves/team` | Yes | Manager, Admin | Get team/all pending leaves |
| `PATCH` | `/api/leaves/:id/approve` | Yes | Manager, Admin | Approve a pending leave |
| `PATCH` | `/api/leaves/:id/reject` | Yes | Manager, Admin | Reject a pending leave |
| `GET` | `/api/assets` | Yes | Any | List assets (role-based filtering) |
| `GET` | `/api/assets/mine` | Yes | Any | Get own assigned assets |
| `POST` | `/api/assets` | Yes | Admin | Create a new asset |
| `PATCH` | `/api/assets/:id` | Yes | Admin | Update an asset |
| `DELETE` | `/api/assets/:id` | Yes | Admin | Delete an asset |
| `PATCH` | `/api/assets/:id/assign` | Yes | Admin | Assign asset to a user |
| `PATCH` | `/api/assets/:id/unassign` | Yes | Admin | Unassign asset from user |
| `GET` | `/api/inventory` | Yes | Admin, Manager | List inventory items |
| `POST` | `/api/inventory` | Yes | Admin | Create inventory item |
| `PATCH` | `/api/inventory/:id` | Yes | Admin | Update inventory item |
| `DELETE` | `/api/inventory/:id` | Yes | Admin | Delete inventory item |
| `PATCH` | `/api/inventory/:id/adjust` | Yes | Admin | Adjust quantity (+1/ΓêÆ1) |
| `GET` | `/health` | No | Any | Health check |

### 6.3 Client Interface

| Aspect | Specification |
|---|---|
| Type | Single Page Application (SPA) |
| Routing | Client-side routing with React Router; protected routes enforce role-based access |
| API Communication | Axios instance with base URL from `VITE_API_URL`; request interceptor adds Bearer token; 401 response clears session and redirects to login |
| State Management | TanStack Query for server state; React Context for auth state |
| Form Handling | React Hook Form with Zod validation schemas |

---

## 7. Constraints

### 7.1 Technical Constraints

| Constraint | Detail |
|---|---|
| Language | TypeScript (strict mode) for both client and server |
| Frontend Framework | React (with Vite as build tool) |
| Backend Framework | Express.js |
| Database | MongoDB with Mongoose ODM |
| Styling | Tailwind CSS |
| Validation | Zod (shared-pattern schemas on both client and server) |
| Runtime | Node.js 20.x |
| Containerisation | Docker with multi-stage builds; Docker Compose for orchestration |
| CI/CD | GitHub Actions |
| Container Registry | GitHub Container Registry (GHCR) |

### 7.2 Operational Constraints

| Constraint | Detail |
| --- | --- |
| No public signup | Users are created exclusively by admins |
| Single organisation | No multi-tenancy support in v1.0 |
| Token storage | JWT stored in localStorage with Axios authorization interceptors (trade-off for simplicity and portable API testing; enterprise revisions can transition to httpOnly cookies) |
| Seed dependency | Demo data requires running `npm run seed`; the seed script refuses to execute in production |
| Free-tier limitations | Optional live deployment may experience cold starts on Render/Vercel free tiers |

### 7.3 Regulatory / Standards Constraints

| Constraint | Detail |
| --- | --- |
| OWASP Top 10 | Basic mitigations applied: input validation (Zod), authentication (JWT + bcrypt), access control (RBAC), security headers (Helmet), rate limiting |
| Data privacy | Password hashes never exposed; no PII logging; `.env` files git-ignored |

---

*End of Software Requirements Specification*